Helm is a tool **used to manage Kubernetes applications**. It **simplifies the process of deploying, configuring, and maintaining complex applications on Kubernetes clusters**. Helm is like a **package manager for Kubernetes** — similar to how `apt` works for Ubuntu or `brew` for macOS, Helm makes it easier to define, install, and upgrade applications on Kubernetes.

At the core of Helm is the concept of **Helm charts**. A **Helm chart is a pre-configured package of Kubernetes resources (like deployments, services, config maps, etc.), which can be easily deployed** to a Kubernetes cluster.

Key Components of Helm:

1. **Helm Client**: The command-line tool we interact with to manage charts and release applications.
2. **Helm Chart**: A collection of files that describe a Kubernetes application, including configuration and dependencies.
3. **Helm Repository**: A location where Helm charts are stored and shared, similar to a software repository.
4. **Helm Release**: A specific instance of a Helm chart deployed to a Kubernetes cluster. When we install a chart, it creates a release.

Why Use Helm?

- **Simplifies deployment**: Instead of writing Kubernetes YAML files for each resource, Helm provides pre-configured charts that we can easily deploy.
- **Version Control**: Helm allows we to manage versions of applications. We can upgrade or roll back to previous versions of an application easily.
- **Configuration Management**: Helm uses values files to manage the configuration of charts, allowing we to customize them for your specific needs.
- **Reusable and Shareable**: Helm charts are reusable and shareable. We can download charts from public repositories or create your own.

How Helm Works

1. **Creating a Helm Chart**: A Helm chart contains several files, with `Chart.yaml` being the primary metadata file. Inside the chart, we may have templates that define Kubernetes resources (like Deployments, Services, ConfigMaps) and values files that specify the configuration parameters for those resources.
2. **Installing a Chart**: When we use Helm to install a chart, it renders the templates with values and sends the resulting Kubernetes YAML to your cluster. It is then deployed as a Kubernetes release.

PATHNEX
Shaping Careers, Building Futures

3. **Upgrading and Managing Releases**: Helm allows we to upgrade your application (Helm release) to a new version of a chart. It can also roll back to a previous version if something goes wrong.

Examples

Step 1: Installing Helm

To start using Helm, first install it on your local machine. We can download the latest version from the official Helm website, or we can use the following commands to install it on various systems:

On macOS:

```
brew install helm
```

On Linux:

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
```

On Windows: We can download the latest release from Helm GitHub releases.

Step 2: Initializing Helm (For Helm v3)

Unlike earlier versions, Helm v3 does not require a Tiller (server-side component) to be installed in the cluster. Just ensure that we have `kubectl` configured and connected to your cluster.

Verify your Helm installation:

```
helm version
```

This should show the version of the Helm client.

Step 3: Adding a Helm Repository

Helm uses repositories to store charts. The default Helm chart repository is `https://charts.helm.sh/stable` (Here - stable is a repository name)

To add the official Helm charts repository:

```
helm repo add stable https://charts.helm.sh/stable
helm repo update
```

PATHNEX
Shaping Careers, Building Futures

Step 4: Installing a Chart

Let's install a simple application using Helm. For example, we will install the **nginx-ingress** controller, a common Kubernetes application.

```
helm install my-nginx-ingress stable/nginx-ingress
```

Here:

- `my-nginx-ingress` is the name of the Helm release.
- `stable/nginx-ingress` is the chart name.

This command will deploy the nginx ingress controller into your Kubernetes cluster. Helm handles all the Kubernetes resources required for the deployment, like deployments, services, and configuration files.

Step 5: Listing Helm Releases

To list all installed Helm releases, use the following command:

```
helm list
```

This will show us all the releases we have deployed to our Kubernetes cluster.

Step 6: Upgrading a Helm Release

If we want to update the version of an application or change its configuration, we can upgrade the release. For example, to upgrade your nginx-ingress release:

```
helm upgrade my-nginx-ingress stable/nginx-ingress
```

Step 7: Rolling Back a Release

If an upgrade doesn't work as expected, we can roll back to a previous version of a release:

```
helm rollback my-nginx-ingress 2
```

This command will roll back the release to revision 2.

Step 8: Uninstalling a Release

To remove a release from your Kubernetes cluster, we can uninstall it:

```
helm uninstall my-nginx-ingress
```

This will delete the resources created by the release.

PATHNEX
Shaping Careers, Building Futures

Practical Example: Customizing a Helm Chart with a Values File

In some cases, we might want to customize the chart's settings, like configuring the replica count or changing the default image for an application. Helm uses a **values file** to specify these settings.

For example, let's customize the **nginx-ingress** chart. First, create a custom `values.yaml` file with the following content:

```
controller:
  replicaCount: 2
  ingressClass: nginx
  service:
    externalTrafficPolicy: Local
```

Then, install the chart using your custom values file:

```
helm install my-nginx-ingress stable/nginx-ingress -f values.yaml
```

Here, you've set the number of replicas to 2 and specified an external traffic policy for the ingress controller service.

PATHNEX
Shaping Careers, Building Futures

Summary

- **Helm** is a powerful tool for managing Kubernetes applications using pre-configured charts.
- It simplifies the deployment, upgrade, and management of Kubernetes applications.
- **Charts** are the main unit of Helm — they define Kubernetes resources, and they can be easily installed and upgraded.
- Helm allows us to share charts, store them in repositories, and use custom values files for configuration.

By using Helm, we can significantly reduce the complexity of deploying and managing Kubernetes applications.

Helm Commands - Summary Notes

- Install: `helm install <release-name> <chart>`
- Upgrade: `helm upgrade <release-name> <chart>`
- Uninstall: `helm uninstall <release-name>`
- List all releases: `helm list`
- Search charts in repo: `helm search repo <chart-name>`
- Show release history: `helm history <release-name>`
- Show chart details: `helm show <chart> [<info-type>]` (info-type: values, readme, chart, all)
- Create a new chart: `helm create <chart-name>`
- Pull chart from repo: `helm pull <chart>` (use `--untar` to extract)
- Dry run install/upgrade: `helm install <release-name> <chart> --dry-run`
  Or
  `helm upgrade <release-name> <chart> --dry-run`
- Render templates without installing: `helm template <release-name> <chart>`
- Uninstall but keep history: `helm uninstall <release-name> --keep-history`
- Add a new repo: `helm repo add <repo-name> <repo-url>`
- Update repo cache: `helm repo update`
- Lint a chart: `helm lint <chart>`
- Package a chart directory: `helm package <chart-directory>`
- Get current values: `helm get values <release-name>` (add `--all` to get all values)
- Get full release details: `helm get all <release-name>`

PATHNEX
Shaping Careers, Building Futures

HELM Chart Directory Structure for “ ”

```
      -chart/
├── .helmignore
├── Chart.yaml
├── values.yaml
├── values-production.yaml
├── values-dev.yaml
├── values-staging.yaml
├── charts/
│   ├── redis-1.0.0.tgz
│   ├── mysql-2.0.0.tgz
│   ├── nginx-3.2.1.tgz
│   └── postgresql-4.1.2.tgz
├── templates/
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── configmap.yaml
│   ├── secret.yaml
│   ├── serviceaccount.yaml
│   ├── hpa.yaml
│   ├── poddisruptionbudget.yaml
│   ├── networkpolicy.yaml
│   ├── persistentvolumeclaim.yaml
│   ├── prometheus-scrape-config.yaml
│   └── cronjob.yaml
├── values/
│   ├── base.yaml
│   ├── dev.yaml
│   ├── prod.yaml
│   └── staging.yaml
├── README.md
├── LICENSE
└── NOTES.txt
```

Breakdown of the Chart Directory

1. ***`Chart.yaml`***

```
apiVersion: v2
name:       -chart
description:         Helm chart for Kubernetes applications
version: 1.0.0
appVersion: 1.0.0
maintainers:
  - name:         Dev Team
    email: dev-team@       .com
dependencies:
  - name: redis
    version: 1.0.0
    repository: "https://charts.bitnami.com/bitnami"
  - name: mysql
    version: 2.0.0
    repository: "https://charts.bitnami.com/bitnami"
```

PATHNEX
Shaping Careers, Building Futures

- This file contains the metadata for the Helm chart. It defines the chart's name (`-chart`), version, description, and any dependencies like Redis or MySQL.

2. ***`values.yaml`*** *(Default Values)*

```
replicaCount: 3

image:
  repository:        /       -app
  tag: latest

service:
  type: ClusterIP
  port: 8080

ingress:
  enabled: true
  host:        .local
  path: /

resources:
  limits:
    cpu: "500m"
    memory: "512Mi"
  requests:
    cpu: "250m"
    memory: "256Mi"

# MySQL configuration (for          app database)
mysql:
  user:        -user
  password:        -password
  database:        -db
```

- This contains the default configuration values for the chart. Students can customize this file or provide environment-specific values using the different `values-<env>.yaml` files.

3. Environment-Specific `values` Files

- **`values-production.yaml`**:

```
replicaCount: 5

image:
  tag: prod-latest

service:
  type: LoadBalancer
  port: 80

resources:
  limits:
    cpu: "1"
    memory: "1Gi"
  requests:
    cpu: "500m"
```

PATHNEX
Shaping Careers, Building Futures

```
    memory: "512Mi"
```

- **values-dev.yaml**:

```
replicaCount: 2

image:
  tag: dev-latest

service:
  type: ClusterIP
  port: 8080

resources:
  limits:
    cpu: "250m"
    memory: "256Mi"
  requests:
    cpu: "100m"
    memory: "128Mi"
```

- **values-staging.yaml**:

```
replicaCount: 3

image:
  tag: staging-latest

service:
  type: ClusterIP
  port: 8081

resources:
  limits:
    cpu: "500m"
    memory: "512Mi"
  requests:
    cpu: "250m"
    memory: "256Mi"
```

PATHNEX
Sharing Careers, Building Futures

These files hold the configuration settings that are specific to different environments (production, development, staging).

*4. **`charts/`** (Dependencies)*

- This folder holds any dependent charts that depends on, like Redis, MySQL, Nginx, etc.
- If relies on Redis, MySQL, or other services, you can bundle the necessary `.tgz` files here. Otherwise, dependencies will be automatically fetched from repositories.

Example:

- **redis-1.0.0.tgz**
- **mysql-2.0.0.tgz**

These `.tgz` files represent the packaged versions of the dependent charts.

*5. **`templates/`** (Kubernetes Resource Templates)*

These templates are used to define the Kubernetes resources that will deploy.
The `.yaml` files inside this directory can be customized to fit the needs of .

- **`_helpers.tpl`**: Contains helper functions for generating reusable templates.

  Example:

  ```
  {{- define "       .labels" -}}
  app: {{ .Release.Name }}
  chart: {{ .Chart.Name }}-{{ .Chart.Version }}
  release: {{ .Release.Name }}
  {{- end -}}
  ```

- **`deployment.yaml`**: Defines the Deployment for the application.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-deployment
  labels:
    {{- include "       .labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
    spec:
      containers:
      - name: {{ .Release.Name }}
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        ports:
        - containerPort: 8080
```

- **`service.yaml`**: Defines the Service to expose the app.

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-service
spec:
  selector:
    app: {{ .Release.Name }}
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 8080
  type: {{ .Values.service.type }}
```

PATHNEX
Shaping Careers, Building Futures

- **`ingress.yaml`**: Defines Ingress for exposing the application via HTTP/S.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ .Release.Name }}-ingress
spec:
  rules:
  - host: {{ .Values.ingress.host }}
    http:
      paths:
      - path: {{ .Values.ingress.path }}
        pathType: Prefix
        backend:
          service:
            name: {{ .Release.Name }}-service
            port:
              number: 8080
```

- **Other templates**:

`configmap.yaml`, **`secret.yaml`**, **`serviceaccount.yaml`**, **`hpa.yaml`**, **`poddisruptionbudget.yaml`**, etc., all work the same way, defining their respective Kubernetes resources for the app.

6. ***`values/`*** *(Environment-Specific Configuration)*

`base.yaml`: General configuration values that apply to all environments.

`dev.yaml`, **`prod.yaml`**, **`staging.yaml`**: These files are specific to each environment, allowing you to configure different settings (e.g., replicas, resources) based on the environment.

7. ***`README.md`***

Documentation for the Helm chart. This file would typically include:

- Instructions on how to install and configure the chart.
- Environment-specific instructions (e.g., for development, production).
- Examples of using `values.yaml` and overriding it with `--set`.

8. ***`LICENSE`***

License under which the Helm chart is distributed.

9. ***`NOTES.txt`***

Post-installation or post-upgrade instructions that can help after deployment of the chart. Example:

```
To access the application, run:

kubectl port-forward svc/{{ .Release.Name }}-service 8080:8080
```

PATHNEX
Shaping Careers, Building Futures